Kaarina Mattika
CS 40
Prof. Kol
6/11/2026

Final Project: Operating System

My final project submission is a guide for installing Arch Linux in preparation for an enterprise environment. The end product of this guide is a user-ready, locally secure, and network-prepared Arch Linux workstation. I have included Linux Unified Key Setup (LUKS) encryption on this workstation, which prepares it for the modern enterprise user that may work in a hybrid or remote role with a laptop.

I have not included SSH hardening or other advanced network security options due to the vendor-saturated network security market, and the likelihood of configuration clashes with third party vendors likely found in an enterprise environment (for example, some tunnel VPN services require their own SSH hardening configurations done through their software). This workstation is being provisioned pre-assignment, so final user definitions will be withheld from some configuration processes.

Usage of the pronouns ‘we’ and ‘our’ in this guide is intended to reference the audience of this document interacting with the author of the document.

Section one: Preparation

We'll begin by downloading a Virtual Machine for this exercise (in this case, Oracle Virtualbox) as well as an .iso of the most recent Arch release from a verified source provided by the distribution's website. In this case, I downloaded it from a mirror provided by the University of Arizona. To ensure that the .iso file has not been tampered with, I checked the SHA checksum provided by the Arch Linux team to the checksum provided by the UoA mirror; they matched (Exhibits 1 - 3).

We'll then move our .iso file into VirtualBox's Virtual Machine folder (to ensure the .iso can be found by others on our team if needed), and load the .iso into VirtualBox. We do not want to proceed with an unattended installation, as we want to both properly configure and secure this workstation. We'll also need to set our virtualized hardware resources to allow our virtualized Arch instance to run with sufficient resources - in this case, I have provisioned 8GB of RAM, 2 CPUs, and 20GB of hard drive space in a Virtual Disk Image (Exhibits 4-6).

Next, we'll ensure the system settings are updated on VirtualBox. We'll **right click** on our Arch machine, navigate to **System,** then **Expert**, and enable UEFI (Exhibit 7). UEFI is the modern firmware bootloader standard, and replaces the legacy BIOS bootloader, which would have been selected for us by default without toggling these settings. While enabling Secure Boot would theoretically give us a more secure installation process on its own, it is unnecessary once we've confirmed the .iso checksum.

Once we complete the pre-configuration of the machine image, we will review the details on the **Machines** tab of VirtualBox. If all of the details look good, we'll then **right click** on our Arch machine, hover over **Start**, and select an option to begin running our VM (Exhibit 8). In this case, we'll select **Start with GUI.**

Section two: Boot

After selecting **Start with GUI,** the Arch boot process will begin. We'll first see the boot splash page, where we can select one of several options to begin our installation. The preferred option, **Arch Linux install medium (x86_64, BIOS),** is already selected for us, so we can hit **Enter** or allow for the boot splash page to time out and select it for us (Exhibit 9). Arch will then perform boot checks and basic installation preparation to ensure the base hardware (in this case, our virtualized hardware) is suitable for Arch (Exhibit 10). Then, we're greeted with Arch's CLI to begin configuring our machine (Exhibit 11).

First, we'll confirm the boot mode - UEFI is the modern firmware standard, and we'll confirm that Arch has booted in UEFI with the command **cat /sys/firmware/efi/fw_platform_size** which should return an expected value of 64, UEFI's bit size (the legacy firmware standard BIOS has a bit size of 16; Exhibit 12). Then we'll want to ensure our Arch instance is connected to the internet - with the commands **ip link** and **ping -c 3 archlinux.org,** we can check both the general networking status of our instance as well as our connectivity to the Arch Linux website. Lastly, we'll enable our system clock and check its status with the commands **timedatectl set-ntp true** and **timedatectl status** (Exhibit 13). The system time will automatically be set to UTC, and we'll set it to our local time zone later.

Section three: Disk Partition

The Arch Wiki recommends an individually partitioned and encrypted container that secures the root filesystem and its swap. Without this setup, the swap can leak memory to disk, defeating the purpose of an encrypted disk setup and leaving the workstation vulnerable to local intrusion. To secure the root filesystem and swap, we'll create a disk partition, create and format a LUKS2 secured container, build a Local Volume Manager inside the container, and then mount the partition to our instance.

First, we'll enter the command **fdisk /dev/sda**, which allows us to begin managing our disk partitions in our virtual memory. The command **g** will create a new partition, which will prompt us for a new disk label, which we'll enter **n** for 'new'. Then, when we're prompted *First Sector*, we'll hit **Enter** and accept the default, and when we're prompted *Last Sector*, we'll input **+1G** to size our partition to 1GB. Now, we'll create a partition with the remaining space on the virtual hard disk by creating a second partition via the inputs **n, 2, enter key, enter key.** We now see that we have two partitions on our virtual disk, partition 1 with 1GB allocated, and partition 2 with 19GB allocated. Then, we'll input **w** to write these changes to disk. (Exhibit 14).

To prepare our root partition, we'll use the command **mkfs.fat -F 32 /dev/sda1** to format our partition "sda1" with the FAT32 file system. Next, we'll format our other partition "sda2" with LUKS2 encryption with the command **cryptsetup luksFormat /dev/sda2.** We're then prompted to create a password for this partition - store this password securely in another machine or service, as without this password, you will not be able to access this partition on this virtualization (Exhibit 15). We'll immediately unlock sda2 with the command **cryptsetup open /dev/sda2 cryptlvm** and your password, in order to set up the Logical Volume Manager (LVM) in sda2.

Then, we'll use the following commands to set up the LVM and their dependent physical and logical volumes:

pvcreate /dev/mapper/cryptlvm
vgcreate vg0 /dev/mapper/cryptlvm
lvcreate -L 4G vg0 -n swap
lvcreate -l 100%FREE vg0 -n root

The first command creates a physical volume allocated to the LVM at the destination file location. The second command creates a volume group at the destination file location, which must match the physical volume, for use by the LVM. The third command creates a 4GB logical volume from the vg0 volume pool and labels it as swap volume, and the fourth command allocates all of the remaining volume to root, ensuring that none of the volume is wasted by remaining unallocated (Exhibit 16).

Next, we'll format these volumes to root and swap with the commands **mkfs.ext4 /dev/vg0/root** and **mkswap /dev/vg0/swap,** respectively. The mkfs.ext4 command creates an EXT4 file system, the default for most Linux systems including Arch. We're using EXT4 for our non-boot files as opposed to the File Allocation Table (FAT) system for our boot files due to its high-performance capabilities, while FAT is ideal for boot files due to its universality. Lastly, we'll mount these volumes with the commands

mount /dev/vg0/root /mnt
mount --mkdir /dev/sda1 /mnt/boot
swapon /dev/vg0/swap

which mount the root, boot, and swap volumes to their respective destinations. We can then verify our disk layout with the command **lsblk** (Exhibit 17).

Section four: Base Arch system configuration

Now that our underlying hard disks are secure, we can begin to install the actual Arch system onto our workstation. Arch uses the unique package manager command **pacman**, as opposed to other Linux package manager commands. Thus, to begin installing Arch, we'll use the command **pacstrap -K /mnt base linux linux-firmware base-devel sudo networkmanager vim man-db man-pages texinfo intel-ucode lvm2.** The **-K** modifier initializes a package signing process inside Arch, which allows for the workstation to begin verifying packages on its own rather than relying on the .iso's initial signing process.

The **/mnt** modifier places all subsequent packages in the command into our secure drive, and the packages themselves (base linux linux-firmware base-devel sudo networkmanager vim man-db man-pages texinfo intel-ucode lvm2). The **base** package is the only required Arch package which includes the pacman utility and directory structure, with the remainder providing required (for our needs) utilities like the linux kernel, the sudo utility, text editors, manpages, and LVM tools (Exhibit 18). Then, we'll generate the system file table, place it in our secure disk, and verify it with the commands **genfstab -U /mnt >> /mnt/etc/fstab** and cat **/mnt/etc/fstab** (Exhibit 19). For the remainder of these steps, we'll jump into the persistent Arch drive with the command **arch-chroot /mnt.** Without doing this, none of our changes will be saved.

Next, we'll set our timezone, hardware clock, locale information, and a root password. With the commands **ln -sf /usr/share/zoneinfo/America/Los_Angeles /etc/localtime** and **hwclock --systohc**, we can set our workstation's timezone to Santa Monica College's timezone. We'll then want to set our workstation's locale with the commands **sed -i 's/^#en_US.UTF-8/en_US.UTF-8/' /etc/locale.gen**, **Locale-gen**, and **echo "LANG=en_US.UTF-8" > /etc/locale.conf**, which will set our workstation to UTF-8 encoding, compatible with nearly all other systems digitally. If we want our workstation to be localized in another language, we can replace 'en' in the third command to another language code, like fr for French or fi for Finnish (Exhibit 20). We'll then set our basic hostname and host file with the commands

```
echo "arch-finalproj-01" > /etc/hostname
cat >> /etc/hosts << 'EOF'
127.0.0.1   localhost
::1         localhost
127.0.1.1   arch-finalproj-01
EOF
```

which sets our default network host information on this workstation (Exhibit 21). Then, we'll set a local password before enabling our network manager and time sync utilities (as unlikely as it is, enabling our network manager before setting a local password means someone could identify our workstation and attack a password unprotected machine) (Exhibit 22).

Section five: HOOKs and Bootloader

This section, while seemingly brief, is critical to successfully getting the Arch virtualization to operate correctly. The *initramfs* is a small system in the /boot folder that bridges the GRUB loader and the full initialization of the system. The *initramfs* is loaded by the GRUB into RAM (alongside the kernel), where it begins the early system setup, before handing it off to the full system. The HOOKs is the order in which these systems are loaded - the order in which the **HOOKS=** line is ordered is the sequence in which *initramfs* initializes early systems. This order is key, because if you load encryption before the LVM, then the disk is encrypted before the LVM can initialize, among endless other problems resulting from endless other misconfigurations. The proper configuration and verification for the HOOKS= line is as follows:

HOOKS=(base udev autodetect microcode modconf kms keyboard keymap consolefont block encrypt lvm2 filesystems fsck)
mkinitcpio -P

The **mkinitcpio -P** command reads the **HOOKS=** config, and proceeds to build the *initramfs* utility in the order it's written (Exhibit 23[1]).

We'll now turn to the bootloader. First, we'll install the GRUB via **pacman -S grub efibootmgr** (Exhibit 24). Then, we'll obtain the UUID of the encrypted partition with **blkid -s UUID -o value /dev/sda2**, make note of the UUID value, and edit */etc/default/* with the following:

GRUB_CMDLINE_LINUX="cryptdevice=UUID=<paste-uuid-here**>:cryptlvm root=/dev/vg0/root"**

An easier version of this with a script (to prevent issues arising from typos) can be done with the command **sed -i "s|^GRUB_CMDLINE_LINUX=.*|GRUB_CMDLINE_LINUX=\"cryptdevice=UUID=$UUID:cryptlvm root=/dev/vg0/root\"|" /etc/default/grub** and verified with the command **grep cryptdevice /etc/default/grub.** We can then finish the GRUB installation with the command **grub-install --target=x86_64-efi**

[1] The first build of this virtualization contained a syntax error that remains present in Exhibit 23. A rebuild of the initramfs and HOOKS utilities was performed via a boot-by-ISO and mounting the encrypted volumes for repair and validation before rebooting. I was unable to capture this process by screenshot.

--efi-directory=/boot --bootloader-id=GRUB and verify the installation with **grub-mkconfig -o /boot/grub/grub.cfg** (Exhibit 25).

Section six: Provisioning user accounts

We'll now create user accounts for our Arch workstation. This is done with the standard **useradd** command. We'll create two user accounts (one administrator with sudoer privileges, and one user account without sudoer privileges) with the following commands:

useradd -m -G wheel -s /bin/bash koladmin
passwd koladmin

useradd -m -s /bin/bash standarduser
passwd standarduser

These commands create two users, one part of the wheel group (and given sudoer privileges), and one not part of the wheel group (without sudoer privileges). Passwords have also been set for each user, with the admin password set to 'adminuser' (as seen in the documentation) (Exhibit 26).

To grant the *wheel* group administrative privileges, we'll need to edit the *visudo* file and delete the "#" sign in the string "# %wheel ALL=(ALL:ALL) ALL" to enable the wheel group's privileges. Then hit the ESC key, type :wq, and hit enter to exit VIM and save changes to the file (Exhibit 27[2]).

We will then reboot the system to ensure all of our initial changes have been successfully written into our Arch virtualization. We exit *chroot*, unmount, and reboot with the commands

exit
umount -R /mnt
reboot

After verifying with our LUKS2 password, we'll verify our users and privileges have been saved (Exhibit 28-29).

[2] This screenshot contains a syntax error with a missing '%' sign. This error was resolved later on when the sudoers file appeared to be broken.

Section seven: Enterprise preparation

Now that we have secured the workstation, provisioned user accounts, and set some basic configurations, we can now begin the final touches to bring this workstation online. As previously stated, the *pacman* utility is Arch's package manager, and we'll use it now to perform the same update and upgrade process other Linux distributions would achieve with **apt-get** with the command **sudo pacman -Syu** (Exhibit 30). We'll also run **sudo pacman -S virtualbox-guest-utils** and **sudo systemctl enable --now vboxservice** to update virtualization specific utility packages (given that we're in a virtualized environment for this exercise, we'll assume the end user is too) and set them to run immediately and persistently with the **--now** command (Exhibit 31).

We are also going to install GNOME to give us a desktop GUI. The following commands will download and enable GNOME on our Arch environment (Exhibit 32-34):

sudo pacman -S gnome gnome-tweaks
sudo systemctl enable gdm
sudo reboot

Now that we have a desktop GUI, let's provide some basic office programs for our end user. LibreOffice is an excellent open-source alternative for Linux, and we'll install the LibreOffice suite with the following command **sudo pacman -S libreoffice-still** (Exhibit 35-36). The **libreoffice-still** option is chosen over **libreoffice-fresh** as the former is the most recent stable, bug-free release, with the latter being more akin to a 'beta' version and less suitable for enterprise environments.

Section eight: Security Hardening

Our last step involves securing the workstation for use in a networked enterprise environment. We'll enable a host firewall, enforce password quality, and set up a basic audit framework. We will allow for more advanced security and networking configurations to be taken into account by the network manager, as previously stated in our executive summary.

First, we'll download, enable, and configure the host firewall with the following commands:

sudo pacman -S ufw
sudo systemctl enable --now ufw
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw enable
sudo ufw status verbose

The first two commands download and enable the host firewall, the third and four commands explicitly deny incoming traffic and permit outgoing traffic, the fifth command starts up the host firewall, and the sixth command confirms our firewall's status (Exhibit 37).

To enforce password quality, we'll use the command **sudo pacman -S libpwquality** to download and install a Pluggable Authentication Module (PAM) in lieu of manually changing user account requirements, which is a more streamlined method of credential management available to enterprise Linux environments (Exhibit 38). We won't configure the PAM fully yet, as we have yet to assign this laptop to an individual user.

Lastly, we'll download and install an auditing module to allow for basic forensic analysis in the case of a breach or user misconduct (Exhibit 39):

sudo pacman -S audit
sudo systemctl enable --now auditd

Section nine: Verification

We want to make sure our final product is properly configured, especially if we plan to use this as our standard configuration image across a fleet of workstations. In addition to the previous checks we've performed (like **sudo ufw status**, **timedatectl**, and **nmcli general status**), three critical final checks should be made before we ship the final product off.

First, we'll check **uname -a** and **pacman -Qu | wc -l,** which provides our kernel version and our 'updated' status. If the first command returns the most recently published .iso or kernel number, and the second command returns the value **0**, then we are all set (Exhibit 40). Secondly, we'll check **nmcli general status** for our connectivity - we want this workstation to be connected to the enterprise environment (which is not always the case, some workstations are intended to be airgapped), and after our check, we see it is connected (Exhibit 41). Lastly, **lsblk -o NAME,TYPE,FSTYPE,MOUNTPOINTS** will give us an overview of our encrypted disk and LVM layers - everything looks to be in order upon inspection (Exhibit 42).

With all of our core configurations confirmed, we are able to mark this project as completed and ship it to the next team for further review if needed, or for deployment into our enterprise environment.